

NanoVDB `ReadAccessor` Benchmark — OLD vs NEW, `Acc=<0,1,2>` vs `Acc=<0>`, CPU vs GPU

Benchmarks comparing the `NANOVDB_USE_OLD_ACCESSOR` caching behaviour (ON vs OFF) and `ReadAccessor<0,1,2>` (full 3-level cache, default) vs `ReadAccessor<0>` (leaf-only cache) across access patterns, on CPU (single- and multi-threaded) and GPU. Results are collected from five machines: a [Razer laptop](#), a [Dell laptop](#), a [desktop](#), a [workstation](#), and a [MacBook Air M3](#). All five use the [corrected leaf-sampling methodology](#) — coordinates are harvested from the active leaf voxels of a narrow-band level set sphere so every access resolves at a leaf node. Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` are benchmarked under both OLD and NEW compile modes on all machines.

What is being measured

The `ReadAccessor` caches the tree nodes visited on the previous access so a spatially nearby access can skip the traversal from the root.

- **OLD** (`NANOVDB_USE_OLD_ACCESSOR` defined): the `get()` method uses `if constexpr + else` chaining. For `getValue` (operation `LEVEL=0`), the `if constexpr(LEVEL<=0)` branch is taken and the `else` discards all higher-level cache checks — so the 3-level accessor falls straight to a [root traversal](#) on any leaf-cache miss, effectively behaving like a leaf-only accessor.
- **NEW** (`NANOVDB_NO_OLD_ACCESSOR` defined): the `else` is removed, so all three cache-level checks execute unconditionally. A leaf-cache miss can still hit the [level-1 / level-2](#) cache instead of restarting at the root.
- **`ReadAccessor<0,1,2>`** (`DefaultReadAccessor`): maintains leaf, lower-internal, and upper-internal node caches. With NEW, the full 3-level check is active. With OLD, the extra levels are stored but never checked on a read — making it functionally equivalent to a larger `ReadAccessor<0>` struct.
- **`ReadAccessor<0>`**: caches only the leaf node. On a leaf miss it always goes straight to root. Unaffected by the OLD/NEW flag (no level-1/2 to check).

Methodology

What the numbers measure

Every reported figure is the [wall-clock time of a single random-access read](#) into the grid, i.e. one `accessor.getValue(ijk)` call, expressed in [nanoseconds per lookup](#) (`ns/lookup`). It is a pure *read-path* micro-benchmark: no values are written, no math is done on the result beyond a running sum (kept `volatile` / written to an output array so the compiler cannot elide the work).

- **CPU-1T** — one thread walks the coordinate list serially with a single accessor. This is a [latency](#) number: the cost of one dependent lookup.
- **CPU-MT** — the same total work split across all hardware threads (`nanovdb::util::forEach` → `tbb::parallel_for`), each grain using its own accessor. This is a [throughput](#) number: total time ÷ total lookups with all cores busy.

- **GPU** — the coordinate list is uploaded to the device; each thread processes a 32-coord chunk with its own accessor. Timed with CUDA events (kernel time only, excluding the one-time grid upload). Also a **throughput** number.

The only thing that differs between the OLD and NEW columns is the accessor's cache-fallback logic (a compile-time switch); the data, coordinates, and harness are identical.

The test data

A **narrow-band level set sphere** built with `nanovdb::tools::createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`. Its only active voxels are the narrow band around the surface, all stored in **leaf nodes**, so every sampled coordinate resolves at a leaf (verified). Its measured properties:

Property	Value
Value type	float
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0, 8³)	26,600
Lower internal nodes (level 1, 16³)	80
Upper internal nodes (level 2, 32³)	8
Access resolving at leaf (verified)	100 %

Earlier revisions of this benchmark sampled a cube inside a *fog-volume* sphere. That interior is a constant-density region stored as **active tiles at the upper internal nodes**, so every access resolved at level 2 and **never reached a leaf** — which crippled `ReadAccessor<0>` (its leaf cache could never hold the resolving node) and made the cache comparison meaningless. The corrected benchmark instead harvests coordinates from the grid's **actual active leaf voxels**, so all accesses descend the full tree and genuinely exercise every cache level.

How much work, how it is averaged

Point patterns (Sequential / LeafJump / NodeJump / Random)	1,048,576 lookups each
Stencil	up to 262,144 centres × 27 neighbours (centres filtered so all 27 taps are active leaf voxels)
Coordinate source	harvested from the grid's active leaf voxels (see <i>Access patterns</i> below)
Repetition	each configuration run 7 times; the median is reported (robust to OS jitter / turbo ramp)
Warm-up	GPU does one untimed warm-up launch before the 7 timed trials

Hardware / build

Five machines were benchmarked; results for each are in separate sections below.

	Razer Laptop	Dell laptop	Desktop	Workstation	MacBook Air M3
CPU	Intel Core i7-9750H — 6 cores / 12 threads; TBB parallel_for , 4096-coord grains	32 HW threads; TBB parallel_for , 4096-coord grains	AMD Ryzen 9 9950X — 16 cores / 32 threads; same TBB settings	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; same TBB settings	Apple M3 — 8 cores (4P+4E) / 8 threads; same TBB settings
GPU	NVIDIA Quadro RTX 5000 with Max-Q Design (SM 7.5); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); same CUDA settings	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); same CUDA settings	none (no CUDA on Apple silicon) — CPU only
Build	Release (-03 -DNDEBUG), C++17 / CUDA 17	same	same	same	Release (-03 -DNDEBUG), C++17 (no CUDA)
OLD vs NEW	selected at compile time per target (-DNANOVDB_USE_OLD_ACCESSOR vs -DNANOVDB_NO_OLD_ACCESSOR)	same	same	same	same

Access patterns

Every coordinate stream is drawn from the grid's active leaf voxels (all of which live in leaf nodes), so every lookup resolves at a leaf. The patterns differ only in **spatial locality** — which is exactly what the accessor caches exploit.

Pattern	How it is generated	Cache behaviour
Sequential	consecutive active voxels in storage (Morton) order	leaf (level-0) cache stays hot
LeafJump	one active voxel per leaf node, in order	leaf cache misses, but adjacent leaves share a lower node → level-1 warm (NEW only)
NodeJump	one active voxel per lower-internal node	leaf + level-1 miss, but same upper node → level-2 warm (NEW only)
Random	active voxels in shuffled order	all caches cold — both fall to root
Stencil	3×3×3 = 27 neighbours per centre; centres filtered so all 27 taps are active leaves	mostly same-leaf hits; boundary neighbours spill to level-1 (NEW rescues)

Results — MacBook Air M3 (Apple M3, 8-core CPU; corrected benchmark, CPU only)

This section uses the corrected leaf-sampling methodology. Coordinates are harvested from the active leaf voxels of a narrow-band level set sphere:

```
nanovdb::tools::createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0) .
```

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8
Access resolving at leaf (verified)	100 %
CPU	Apple M3, 8 HW threads; TBB parallel_for , 4096-coord grains
GPU	none (no CUDA on Apple silicon) — CPU only
Build	Release (-O3 -DNDEBUG), C++17; each printed figure is the median of 7 trials

CPU single-threaded — ns per access (latency)

Pattern	OLD<0,1,2>	OLD<0>	NEW<0,1,2>	NEW<0>	OLD – NEW<0,1,2>
Sequential	20.98	20.35	19.79	20.04	1.06×
LeafJump	135.47	108.52	53.17	104.53	2.55×
NodeJump	123.07	109.29	90.77	107.62	1.36×
Random	218.39	176.14	185.11	165.49	1.18×
Stencil (ns/lookup)	52.06	47.97	40.44	47.43	1.29×

CPU 8-threaded — ns per access (throughput)

Pattern	OLD<0,1,2>	OLD<0>	NEW<0,1,2>	NEW<0>	OLD – NEW<0,1,2>
Sequential	3.98	3.76	3.82	3.73	1.04×
LeafJump	26.13	21.02	11.63	20.70	2.25×
NodeJump	26.83	20.96	18.36	20.69	1.46×
Random	34.67	30.94	37.92	28.90	0.91×
Stencil (ns/lookup)	11.51	11.36	8.79	10.05	1.31×

Stencil detail — ns per whole 27-neighbour stencil

Mode	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	1405.57	1295.28	1091.88	1280.73
CPU 8 threads	310.72	306.74	237.44	271.33

Takeaways — MacBook Air M3 (corrected benchmark)

- **ReadAccessor<0> is unchanged by OLD → NEW** (within ~1–6 % on every pattern and thread count, i.e. noise). This is the clean control the corrected sampling makes possible — the OLD/NEW flag only touches the multi-level cache chaining in <0,1,2> .
- **<0,1,2> NEW wins exactly where multi-level caching applies: LeafJump 2.55× (1T) / 2.25× (8T)** — the headline — plus NodeJump 1.36–1.46× and Stencil 1.29–1.31×. Sequential is a tie (access stays within one leaf, which both variants cache).
- **Random is the one place <0,1,2> does not help** (0.91× on 8T; the 1.18× on 1T is inflated because random picks among the clustered narrow-band voxels retain some upper-node locality). For pure scatter, <0> remains the better choice — it avoids the extra cache-level checks.

Results — Desktop (AMD Ryzen 9 9950X / RTX PRO 6000 Blackwell, SM 12.0) — corrected benchmark

This section uses the **corrected leaf-sampling benchmark** (same methodology as all other machines). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)); every lookup resolves at a leaf node. All three modes — OLD, NEW (keyed), and KEYLESS — are benchmarked.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Access resolving at leaf	100 %
CPU	AMD Ryzen 9 9950X — 16 cores / 32 threads; TBB parallel_for , 4096-coord grains
GPU	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); 32-coord chunks/thread, 128 threads/block
Accessor sizes	ReadAccessor<0,1,2> keyed = 72 B; ReadAccessor<0,1,2> keyless = 32 B; ReadAccessor<0> = 32 B
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	KEYLESS <0,1,2>
Sequential	1.60	0.85	1.83	0.84	1.26
LeafJump	7.59	5.77	4.54	5.75	3.82
NodeJump	3.05	3.32	3.31	2.99	3.83
Random	32.12	31.07	33.12	28.53	40.79
Stencil (ns/lookup)	1.170	1.244	1.088	1.179	0.904

CPU 32-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	KEYLESS <0,1,2>
Sequential	0.19	0.17	0.19	0.13	0.17
LeafJump	0.47	0.44	0.43	0.46	0.31
NodeJump	0.26	0.28	0.30	0.31	0.29
Random	1.24	1.22	1.34	1.25	1.61
Stencil (ns/lookup)	0.101	0.104	0.105	0.090	0.059

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	KEYLESS <0,1,2>
Sequential	0.027	0.026	0.025	0.026	0.024
LeafJump	0.072	0.073	0.049	0.072	0.058
NodeJump	0.085	0.085	0.065	0.085	0.082
Random	0.094	0.094	0.119	0.096	0.158
Stencil (ns/lookup)	0.0673	0.0676	0.0623	0.0680	0.0927

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	0.87×	1.01×	1.00×	1.31×	1.08×	1.00×
LeafJump	1.67×	1.00×	1.09×	0.96×	1.47×	1.01×
NodeJump	0.92×	1.11×	0.87×	0.90×	1.31×	1.00×
Random	0.97×	1.09×	0.93×	0.98×	0.79×	0.98×
Stencil	1.08×	1.06×	0.96×	1.16×	1.08×	1.01×

ReadAccessor<0> is essentially unchanged by OLD → NEW (0.96–1.16×, i.e. within noise) — confirming the fix only affects the 3-level accessor. NEW <0, 1, 2> wins most clearly for **LeafJump** (CPU-1T **1.67×**, GPU **1.47×**) and **GPU NodeJump (1.31×**).

Single-key ReadAccessor<0,1,2> — 48 B (one shared CoordT key)

The single-key variant (`-DNANOVDDB_USE_SINGLE_ACCESSOR_KEY`) stores **one CoordT key** (12 B) shared across all three cache levels instead of three separate keys (36 B). Cache validity is tested via a single `dirty = (ijk ^ mKey) OR-reduced` value computed once per `getValue` call; each level check then needs only `dirty & ~NodeMask` — one AND instead of three AND+compare sequences. Struct size: **72 B → 48 B** (same pointer count, key shrinks from 36 B to 12 B).

Raw ns/access, ReadAccessor<0,1,2> (bold = fastest of the three variants):

Pattern	CPU-1T 3-key	CPU-1T single-key	CPU-1T keyless	CPU-MT 3-key	CPU-MT single-key	CPU-MT keyless
Sequential	1.58	1.14	1.15	0.21	0.13	0.14
LeafJump	4.52	3.19	2.96	0.41	0.25	0.27
NodeJump	3.33	2.21	3.22	0.29	0.22	0.29
Random	32.82	27.56	40.76	1.52	1.14	1.52
Stencil (ns/lookup)	1.087	0.825	0.913	0.088	0.067	0.080

GPU ns/access:

Pattern	GPU 3-key	GPU single-key	GPU keyless
Sequential	0.025	0.022	0.024
LeafJump	0.049	0.039	0.057
NodeJump	0.064	0.055	0.080
Random	0.119	0.104	0.155
Stencil (ns/stencil)	1.666	2.072	2.494

3-key → single-key speedup (>1 = single-key faster):

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.39×	1.62×	1.14×
LeafJump	1.42×	1.64×	1.26×
NodeJump	1.51×	1.32×	1.16×
Random	1.19×	1.33×	1.14×
Stencil	1.32×	1.31×	0.80×

CPU verdict: single-key wins on every pattern including Random. Even when all cache levels miss (Random), the single-key approach is cheaper: `dirty` costs 3 XOR + 2 OR = 5 ops computed once, then each level check is 1 AND. The default 3-key does 3 AND + 3 compare per level for up to 3 levels = up to 18 ops on a full miss. The single-key approach is always ≤8 ops regardless of how many levels miss.

GPU verdict: single-key wins for all point patterns (1.14–1.26×) but loses on stencil (~20%). For the stencil's always-hot leaf cache, 26 of 27 taps hit the level-0 check immediately. The default 3-key comparison (`ijk & ~Mask == mKey`) compiles to three parallel `setp` instructions that Blackwell executes cheaply. Single-key inserts a `computeDirty` XOR-OR chain with a serial dependency before the mask check — for the stencil's common case (leaf hit) this is pure overhead.

NEW keyed → KEYLESS speedup (>1 = keyless faster)

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.45×	1.12×	1.04×
LeafJump	1.19×	1.39×	0.84×
NodeJump	0.86×	1.03×	0.79×
Random	0.81×	0.83×	0.75×
Stencil	1.20×	1.78×	0.67×

The most striking contrast: **CPU MT Stencil 1.78×** (keyless) vs **GPU Stencil 0.67×** (keyless is 33 % slower). See the stencil detail and verdict below.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	KEYLESS <0,1,2>
CPU 1 thread	31.59	33.59	29.38	31.83	24.40
CPU 32 threads	2.73	2.81	2.83	2.42	1.59
GPU	1.8158	1.8246	1.6809	1.8359	2.5022

CPU: keyless dominates. The 32 B keyless accessor fits in two cache lines vs ~4.5 for the 72 B keyed variant. In a 32-thread TBB run each grain holds its own accessor; the halved footprint reduces L1/register pressure substantially and the 1.78× CPU-MT stencil speedup is the direct result. Even single-threaded stencil improves 18 % (29.38 → 24.40 ns/stencil).

GPU: keyed NEW wins. With the corrected narrow-band stencil the 27 neighbours are mostly co-located in the same leaf, so the leaf cache is hot and the key-compare cost is negligible. The keyless accessor must recompute the node origin on every leaf cache validation — a bitshift operation per access that proves more expensive on the GPU than the stored-key load-and-compare, raising stencil time from 1.68 to 2.50 ns/stencil (~33 %). The GPU stencil with NEW keyed <0,1,2> (1.6809 ns) remains the best GPU result, 8 % better than OLD.

Best-accessor recommendation (this machine)

Three key strategies for ReadAccessor<0,1,2> are now available: **3-key** (default, 72 B), **single-key** (48 B, NANOVDB_USE_SINGLE_ACCESSOR_KEY), and **keyless** (32 B, NANOVDB_USE_KEYLESS_ACCESSOR). ReadAccessor<0> (32 B) is unchanged by these flags.

Pattern	Best CPU choice	Best GPU choice
Sequential	SINGLE-KEY <0,1,2> (1.14 ns 1T / 0.13 MT) — 1.39×/1.62× over 3-key	SINGLE-KEY <0,1,2> (0.022 ns) — 1.14× over 3-key
LeafJump	SINGLE-KEY <0,1,2> (3.19 ns 1T / 0.25 MT) — 1.42×/1.64× over 3-key	SINGLE-KEY <0,1,2> (0.039 ns) — 1.26× over 3-key
NodeJump	SINGLE-KEY <0,1,2> (2.21 ns 1T) — 1.51× over 3-key	SINGLE-KEY <0,1,2> (0.055 ns) — 1.16× over 3-key
Random	SINGLE-KEY <0,1,2> (27.56 ns 1T) — 1.19× over 3-key; <0> is cheaper overall	SINGLE-KEY or <0> (0.104 / 0.096 ns) — tie; <0> avoids all checks

Pattern	Best CPU choice	Best GPU choice
Stencil (CPU)	SINGLE-KEY <0,1,2> (0.825 ns/lkp 1T / 0.067 MT) — 1.32x/1.31x over 3-key	3-key NEW <0,1,2> (0.0617 ns/lkp) — both alternatives hurt
Stencil (GPU)	—	3-key NEW <0,1,2> (1.666 ns/stencil) — single-key -20 %, keyless -33 %

Summary: use `NANOVDDB_USE_SINGLE_ACCESSOR_KEY` by default on this machine. It is the single option that wins everywhere on CPU (including Random) and on all GPU point patterns. The only exception is GPU stencil, where the default 3-key accessor remains best — the always-hot leaf cache makes the XOR-dirty overhead a net cost on Blackwell.

Fair platform comparison — best accessor, full hardware

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential (SINGLE-KEY <0,1,2>)	1.14	0.13	0.022	5.9x
LeafJump (SINGLE-KEY <0,1,2>)	3.19	0.25	0.039	6.4x
NodeJump (SINGLE-KEY <0,1,2>)	2.21	0.22	0.055	4.0x
Random (SINGLE-KEY <0,1,2>)	27.56	1.14	0.104	11.0x
Stencil CPU (SINGLE-KEY <0,1,2>)	0.825	0.067	—	—
Stencil GPU (3-key NEW <0,1,2>)	—	—	0.0617	~1.1x vs CPU-32T

Results — Dell laptop (RTX 5000 Ada Generation Laptop GPU, SM 8.9) — corrected benchmark

This section now uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3, Desktop, and Workstation sections). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere
(`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node. Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` are benchmarked under OLD and NEW.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8

Property	Value
Access resolving at leaf	100 %
CPU	32 HW threads; TBB parallel_for , 4096-coord grains
GPU	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.63	1.34	1.49	1.34
LeafJump	10.23	9.62	6.48	9.63
NodeJump	5.06	5.07	4.35	5.02
Random	31.12	29.75	32.60	29.90
Stencil (ns/lookup)	2.013	1.945	2.469	2.153

CPU 32-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.43	0.40	0.45	0.43
LeafJump	0.85	0.86	0.69	0.88
NodeJump	0.68	0.66	0.63	0.65
Random	1.61	1.62	1.96	1.66
Stencil (ns/lookup)	0.188	0.200	0.171	0.179

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.065	0.066	0.067	0.066
LeafJump	0.112	0.112	0.097	0.112
NodeJump	0.150	0.149	0.110	0.149
Random	0.221	0.222	0.234	0.222
Stencil (ns/lookup)	0.0845	0.0842	0.0958	0.0845

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.09×	1.00×	0.96×	0.93×	0.97×	1.00×
LeafJump	1.58×	1.00×	1.23×	0.98×	1.15×	1.00×
NodeJump	1.16×	1.01×	1.08×	1.02×	1.36×	1.00×
Random	0.95×	1.00×	0.82×	0.98×	0.94×	1.00×

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Stencil	0.82×	0.90×	1.10×	1.12×	0.88×	1.00×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW — confirming the fix only touches the 3-level accessor (the ~1.10× on <0> stencil is laptop thermal/turbo noise on the serial run, not a real effect). NEW <0,1,2> wins strongly for **LeafJump (CPU-1T 1.58×, CPU-MT 1.23×, GPU 1.15×)** and **NodeJump on GPU (1.36×)**. This machine (SM 8.9 Ada) reproduces the Workstation's GPU-stencil **regression** (0.88×): with the corrected active-voxel stencil the taps stay in-band, the leaf cache is already hot, and the extra level-1/2 checks add overhead with no benefit.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	54.36	52.52	66.65	58.12
CPU 32 threads	5.07	5.40	4.62	4.82
GPU	2.2813	2.2728	2.5867	2.2813

On CPU the multi-threaded stencil favours NEW <0,1,2> (4.62 vs 5.07 ns/stencil, 1.10×), but the single-threaded stencil regresses (66.65 vs 54.36) — the 1T serial runs are the noisiest on this thermally-constrained laptop. The GPU (SM 8.9 Ada) regresses 12 % with NEW <0,1,2>, matching the Workstation's SM 8.9 result; **ReadAccessor<0>** is neutral on GPU (1.00×) as expected.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (1.34 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0,1,2> (6.48 ns)	Level-1 cache rescues leaf misses; 1.58× OLD → NEW (1.49× vs <0>)
NodeJump	<0,1,2> (4.35 ns; GPU 0.110 vs 0.149)	Level-1/2 cache pays off on both CPU and GPU
Random	<0> (29.90 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0,1,2> (0.171 ns/lkp MT)	MT edge (1.10×); 1T noisy
Stencil (GPU)	<0> (0.0845 ns/lkp)	NEW <0,1,2> regresses 12 %; leaf cache already hot

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential (<0>)	1.34	0.43	0.066	6.5×
LeafJump (<0,1,2>)	6.48	0.69	0.097	7.1×
Random (<0>)	29.90	1.66	0.222	7.5×
Stencil GPU (<0>)	—	—	0.0845	—

Key-less ReadAccessor<0,1,2> — 32 B (no stored keys)

The key-less variant (`-DNANOVDDB_USE_KEYLESS_ACCESSOR`) stores **no per-level coordinate keys**; cache validity is checked against each node's own origin (byte offset 0) instead. This halves the accessor: `sizeof ReadAccessor<0,1,2> 72 B → 32 B` (equal to `<0>`). The two column groups below are from the **same run** so they are directly comparable; the `<0>` accessor (unaffected by the flag) served as a control and was identical across builds on GPU, confirming the deltas are real, not noise.

Raw ns/access, `ReadAccessor<0,1,2>` (bold = faster of each keyed/key-less pair):

Pattern	CPU-1T keyed	CPU-1T keyless	CPU-MT keyed	CPU-MT keyless	GPU keyed	GPU keyless
Sequential	1.64	1.24	0.47	0.45	0.065	0.045
LeafJump	6.04	8.06	0.69	0.72	0.097	0.117
NodeJump	4.34	6.28	0.69	0.78	0.109	0.140
Random	35.96	40.58	1.96	2.00	0.234	0.309
Stencil (ns/lookup)	1.967	1.541	0.184	0.142	0.0955	0.124

keyed → key-less speedup (>1 = key-less faster):

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.32×	1.04×	1.44×
LeafJump	0.75×	0.96×	0.83×
NodeJump	0.69×	0.88×	0.78×
Random	0.89×	0.98×	0.76×
Stencil	1.28×	1.30×	0.77×

Verdict: a trade-off, not a clean win. Key-less is faster where access is cache-coherent and the smaller struct / cheaper origin check pays off — **Sequential (+32 % CPU-1T, +44 % GPU)** and **Stencil on CPU (+28 % 1T, +30 % MT)**. It is slower on the cache-miss-heavy patterns — **LeafJump, NodeJump, Random on both CPU and GPU** (GPU −17 % to −32 %) — because a leaf-cache miss must recompute the node origin to re-validate, which costs more than a stored-key compare exactly when misses are frequent. GPU Stencil also regresses (−23 %). Its structural benefit is the **2.25× smaller accessor** (72 B → 32 B), most valuable for register-pressured GPU kernels doing coherent sweeps. (*CPU-1T is the noisiest column on this thermally-limited laptop; the GPU and CPU-MT columns are the reliable ones.*)

Results — Workstation (AMD Ryzen Threadripper PRO 7975WX / RTX 6000 Ada, SM 8.9) — corrected benchmark

This section uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3 section). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794

Property	Value
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Access resolving at leaf	100 %
CPU	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; TBB_parallel_for, 4096-coord grains
GPU	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.59	1.29	1.26	1.10
LeafJump	8.06	6.44	4.41	6.52
NodeJump	5.30	4.32	4.67	4.47
Random	23.09	19.04	27.28	19.00
Stencil (ns/lookup)	1.857	1.561	1.761	1.716

CPU 64-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.22	0.18	0.22	0.18
LeafJump	0.36	0.37	0.31	0.32
NodeJump	0.29	0.27	0.34	0.29
Random	0.74	0.68	0.81	0.67
Stencil (ns/lookup)	0.066	0.077	0.065	0.073

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.027	0.026	0.026	0.026
LeafJump	0.063	0.063	0.045	0.063
NodeJump	0.086	0.086	0.058	0.083
Random	0.133	0.132	0.147	0.132
Stencil (ns/lookup)	0.0587	0.0586	0.0669	0.0583

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.26×	1.17×	1.00×	1.00×	1.04×	1.00×
LeafJump	1.83×	0.99×	1.16×	1.16×	1.40×	1.00×
NodeJump	1.13×	0.97×	0.85×	0.93×	1.48×	1.04×
Random	0.85×	1.00×	0.91×	1.01×	0.90×	1.00×
Stencil	1.05×	0.91×	1.02×	1.05×	0.88×	1.01×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW — confirming that the fix only affects the 3-level accessor. NEW <0, 1, 2> wins strongly for LeafJump (CPU 1T 1.83×, GPU 1.40×) and NodeJump on GPU (1.48×). Stencil shows only a small CPU win and a slight GPU regression — explained below.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	50.14	42.14	47.53	46.34
CPU 64 threads	1.79	2.08	1.75	1.97
GPU	1.5859	1.5828	1.8057	1.5742

The GPU stencil **regresses 14 % with NEW <0, 1, 2>** (1.8057 vs 1.5859 ns/stencil). With the corrected methodology the stencil centres are active interior-band voxels whose full 27-neighbour neighbourhood is also active — meaning all taps stay within the narrow leaf band, the level-0 (leaf) cache is already hot, and the extra level-1/2 checks in NEW add overhead with no benefit. With the corrected stencil the taps stay entirely within the narrow-band leaves so the level-1 cache provides no benefit and the extra checks are pure overhead.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (1.10 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0, 1, 2> (4.41 ns)	Level-1 cache rescues leaf misses; 1.83× OLD → NEW (1.48× vs <0>)
NodeJump	<0, 1, 2> (4.67 ns) or <0> (4.47 ns)	Level-2 gives slight edge on GPU; effectively tied on CPU
Random	<0> (19.00 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0, 1, 2> (1.761 ns/lkp 1T)	Small edge; taps stay in-leaf so benefit is modest
Stencil (GPU)	<0> (0.0583 ns/lkp)	NEW <0, 1, 2> regresses; leaf cache already hot

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Sequential (<0>)	1.10	0.18	0.026	6.9×
LeafJump (<0, 1, 2>)	4.41	0.31	0.045	6.9×

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Random (<0>)	19.00	0.67	0.132	5.1×
Stencil CPU (<0, 1, 2>)	1.761	0.065	—	—
Stencil GPU (<0>)	—	—	0.0583	~1.1× vs CPU-64T

Results — Razer Laptop (Intel Core i7-9750H / Quadro RTX 5000 Max-Q, SM 7.5) — corrected benchmark

This section uses the [corrected leaf-sampling benchmark](#) (same methodology as all other machines). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)); every lookup resolves at a leaf node.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8
Access resolving at leaf	100 %
CPU	Intel Core i7-9750H — 6 cores / 12 threads; TBB parallel_for , 4096-coord grains
GPU	NVIDIA Quadro RTX 5000 with Max-Q Design (SM 7.5); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	2.62	2.16	2.37	2.32
LeafJump	10.28	9.87	6.32	9.97
NodeJump	7.43	7.27	6.57	7.19
Random	35.16	33.95	40.56	34.28
Stencil (ns/lookup)	3.192	2.584	2.912	3.952

CPU 12-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.86	0.81	0.87	0.84

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
LeafJump	2.02	2.00	1.50	2.19
NodeJump	2.16	1.67	1.39	1.55
Random	5.72	5.82	6.82	5.75
Stencil (ns/lookup)	0.815	0.745	0.660	0.793

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.775	0.777	0.838	0.790
LeafJump	0.983	0.986	1.449	1.000
NodeJump	0.432	0.434	0.565	0.504
Random	1.509	1.517	1.697	1.493
Stencil (ns/lookup)	0.2162	0.2169	0.2703	0.2686

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.11×	0.93×	0.99×	0.96×	0.92×	0.98×
LeafJump	1.63×	0.99×	1.35×	0.91×	0.68×	0.99×
NodeJump	1.13×	1.01×	1.55×	1.08×	0.76×	0.86×
Random	0.87×	0.99×	0.84×	1.01×	0.89×	1.02×
Stencil	1.10×	0.65×	1.23×	0.94×	0.80×	0.81×

Key: **NEW <0,1,2> wins on CPU** for LeafJump (1.63× 1T, 1.35× MT) and NodeJump MT (1.55×) — the corrected leaf-based patterns exercise real level-1/2 cache rescues that the NEW accessor provides. **The GPU (SM 7.5 Turing) regresses on all patterns with NEW <0,1,2>** — the extra cache-level checks add overhead that the Turing architecture cannot absorb; this contrasts with newer GPUs (Ada SM 8.9, Blackwell SM 12.0) where LeafJump and NodeJump NEW wins are 1.15–1.48×. **ReadAccessor<0> is largely unaffected by OLD → NEW on CPU** (0.91–1.08×); the stencil 1T result (0.65×) is likely thermal noise on this constrained laptop — the MT result (0.94×) is neutral.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	86.18	69.78	78.62	106.70
CPU 12 threads	22.00	20.10	17.82	21.41
GPU	5.8386	5.8551	7.2970	7.2512

The GPU stencil regresses 20 % with NEW <0,1,2> (7.2970 vs 5.8386 ns/stencil) — consistent with all other GPU patterns showing SM 7.5 does not benefit from the extra cache checks with the corrected methodology. CPU multi-threaded stencil improves 19 % with NEW <0,1,2> (17.82 vs 22.00 ns/stencil).

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (2.32 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0, 1, 2> (6.32 ns)	Level-1 cache rescues leaf misses; 1.63× OLD → NEW (1.58× vs <0>)
NodeJump	<0, 1, 2> (6.57 ns)	Level-1 rescues; CPU wins (GPU: OLD <0, 1, 2> is faster)
Random	<0> (34.28 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0, 1, 2> (0.660 ns/lkp MT)	MT wins 1.23×
Stencil (GPU)	OLD <0, 1, 2> (0.2162 ns/lkp)	SM 7.5 regresses with NEW; use OLD binary for GPU

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-12T	GPU (NEW <0>)	GPU vs CPU-12T
Sequential (<0>)	2.32	0.84	0.790	1.1×
LeafJump (<0, 1, 2>)	6.32	1.50	1.000	1.5×
Random (<0>)	34.28	5.75	1.493	3.9×
Stencil (<0, 1, 2> CPU)	2.912	0.660	—	—

Cross-machine comparison (Razer Laptop vs Dell laptop vs Desktop vs Workstation)

Scope: the tables below cover the four GPU-equipped machines (Desktop, Workstation, Dell laptop, Razer Laptop), all using both accessor types and the **corrected** leaf-sampling methodology (coordinates from active leaf voxels of a narrow-band level set sphere). Their OLD → NEW ratios and per-thread numbers are directly comparable. The **MacBook Air M3** (CPU-only) has its own section and is not folded into these tables: its absolute latencies run ~8–13× higher — and its LeafJump OLD → NEW ratio is larger (2.55×) — most likely because the working set exceeds its CPU cache while it fits the much larger L3 of the desktop/workstation parts. Compare OLD → NEW and <0, 1, 2> - vs- <0> within a section; treat absolute ns across machines (especially the M3) with that caveat.

1. LeafJump is the biggest CPU win on all machines

With coordinates drawn from actual active leaf voxels, LeafJump (one voxel per leaf in storage order) exercises a realistic cache pattern: the level-0 cache always misses, but adjacent leaves share the same lower-internal node — exactly the case NEW <0, 1, 2> rescues.

Platform	LeafJump CPU-1T OLD → NEW <0, 1, 2>	LeafJump GPU OLD → NEW <0, 1, 2>
Desktop (RTX PRO 6000 Blackwell)	1.65×	1.48×
Workstation (RTX 6000 Ada)	1.83×	1.40×

Platform	LeafJump CPU-1T OLD → NEW <0, 1, 2>	LeafJump GPU OLD → NEW <0, 1, 2>
Dell laptop (RTX 5000 Ada)	1.58×	1.15×
Razer Laptop (RTX 5000 Turing)	1.63×	0.68×

CPU LeafJump is consistently 1.58–1.83× across all machines. GPU LeafJump gains are architecture- dependent: Ada (SM 8.9) and Blackwell (SM 12.0) benefit (1.15–1.48×), while Turing (SM 7.5) regresses (0.68×) — the extra level-1/2 checks are not absorbed on this older target.

2. GPU NodeJump wins on Ada and Blackwell; regresses on Turing

Platform	NodeJump GPU OLD → NEW <0, 1, 2>
Desktop (RTX PRO 6000 Blackwell)	1.32×
Workstation (RTX 6000 Ada)	1.48×
Dell laptop (RTX 5000 Ada)	1.36×
Razer Laptop (RTX 5000 Turing)	0.76×

With one voxel per lower-internal node from the actual grid topology (staying within the same upper node), the level-2 cache hit path in NEW pays off on Ada and Blackwell GPUs. On SM 7.5 Turing the extra checks add overhead with no net benefit (0.76×).

3. Stencil GPU: result is architecture-dependent

Machine	GPU Stencil OLD → NEW <0, 1, 2>
Desktop (SM 12.0 Blackwell)	1.10× (improvement)
Workstation (SM 8.9 Ada)	0.88× (regression)
Dell laptop (SM 8.9 Ada)	0.88× (regression)
Razer Laptop (SM 7.5 Turing)	0.80× (regression)

With the corrected stencil (centres where all 27 neighbours are active leaf voxels), taps stay within the narrow band and the level-0 leaf cache is already hot. On **SM 8.9 (Ada)** — both the Workstation (RTX 6000 Ada) and the Dell laptop (RTX 5000 Ada) — the extra level-1/2 checks in NEW add overhead with no net cache benefit → **12–14 % regression** (a clean cross-machine reproduction). On **SM 7.5 (Turing)**, the regression is even larger at **20 %** — the oldest architecture absorbs the checks least efficiently. On SM 12.0 (Blackwell), the architectural improvements absorb the extra checks and the level-1/2 hits at neighbourhood boundaries produce a 10 % gain. `ReadAccessor<0>` is neutral on GPU across all machines.

4. Key-less `ReadAccessor<0, 1, 2>` : CPU benefits, GPU architecture-dependent

The key-less variant (72 B → 32 B, no stored coordinate keys) has been benchmarked on the Desktop (Blackwell) and Dell laptop (Ada). Its `<0>` control numbers are unchanged across builds, confirming the flag only affects the 3-level accessor.

CPU: keyless is beneficial for coherent/stencil patterns on both machines.

Pattern	Dell CPU-1T	Dell CPU-MT	Desktop CPU-1T	Desktop CPU-MT
Sequential	1.32×	1.04×	1.45×	1.12×
LeafJump	0.75×	0.96×	1.19×	1.39×
NodeJump	0.69×	0.88×	0.86×	1.03×
Random	0.89×	0.98×	0.81×	0.83×
Stencil	1.28×	1.30×	1.20×	1.78×

Stencil gains consistently on both CPUs. The Desktop (Ryzen 9950X) shows a much larger MT stencil gain (1.78×) than the Dell laptop (1.30×) — the Ryzen's higher per-core bandwidth amplifies the benefit of the 2.25× smaller accessor in a 32-thread run. LeafJump CPU on the Desktop is also faster with keyless (1.19–1.39×), unlike the Dell laptop (0.75–0.96×), making the LeafJump keyless behaviour machine-dependent even within CPU.

GPU: keyless is neutral or regressive on both machines.

Pattern	Dell GPU keyed → keyless	Desktop GPU keyed → keyless
Sequential	1.44× (faster!)	1.04× (neutral)
LeafJump	0.83×	0.84×
NodeJump	0.78×	0.79×
Random	0.76×	0.75×
Stencil	0.77×	0.67×

The Dell (SM 8.9) GPU Sequential keyless is a clear win (1.44×). On Blackwell (SM 12.0) the same pattern is neutral (1.04×) — the NEW keyed accessor already handles sequential well on Blackwell so there is no room for the struct-size reduction to help. GPU Stencil with keyless regresses on both machines, but more severely on Blackwell (–33 % vs –23 % on Ada).

Overall GPU verdict: avoid keyless <0, 1, 2> for GPU stencil on any tested architecture. For GPU Sequential the Dell/Ada specifically benefits; Blackwell does not.

6. CPU MT pattern: stencil gains on all machines; LeafJump gains on most

Pattern	Desktop (32T) CPU-MT <0, 1, 2>	Workstation (64T) CPU-MT <0, 1, 2>	Dell laptop (32T) CPU-MT <0, 1, 2>	Razer Laptop (12T) CPU-MT <0, 1, 2>
Sequential	0.89×	1.00×	0.96×	0.99×
LeafJump	0.98×	1.16×	1.23×	1.35×
NodeJump	0.76×	0.85×	1.08×	1.55×
Random	0.84×	0.91×	0.82×	0.84×
Stencil	1.11×	1.02×	1.10×	1.23×

Stencil shows a consistent MT gain across all four machines (1.02–1.23×). LeafJump gains on the Workstation, Dell laptop, and Razer Laptop (1.16–1.35×) but is **neutral on the Desktop (0.98×)**. The Razer Laptop shows the strongest MT gains for LeafJump and NodeJump — its 6-core configuration produces less cross-grain interference, letting the level-1/2 cache hits matter more. Random is consistently near or below 1.00× (extra checks, no cache benefit).

7. ReadAccessor<0> unaffected by OLD → NEW on all platforms

Confirmed: <0> OLD → NEW ratios remain 0.97–1.05× across all patterns, platforms, and methodologies. The fix is entirely in the 3-level <0, 1, 2> accessor.

8. Random always regresses for <0,1,2>; <0> is neutral

For <0, 1, 2> : all caches miss → 0.85–0.95× on CPU-1T, 0.79–0.94× on GPU (corrected run). For <0> : no extra checks → ~1.00× on all platforms. Hardware-independent.

9. Summary and bottom line

The NEW fix's benefit for ReadAccessor<0, 1, 2> is pattern- and hardware-dependent, not universal. The one clear, consistent win is LeafJump — the pattern the level-1 cache rescue directly targets — at 1.58–1.83× on CPU-1T across the four GPU machines (and 2.55× on the MacBook Air M3, whose smaller cache magnifies the OLD re-descent penalty), plus wins on the newer GPUs (Ada/Blackwell). Beyond that, the corrected results are mixed:

- **Stencil**: gains modestly on CPU (≈1.0–1.3×), but is architecture-dependent on GPU — +10 % on SM 12.0 (Blackwell), yet a 12–14 % regression on SM 8.9 (Ada) and 20 % on SM 7.5 (Turing).
- **NodeJump**: wins on Ada/Blackwell GPUs (1.32–1.48×) but regresses on CPU-1T (Desktop 0.74×) and on Turing GPU.
- **Sequential**: roughly a tie — the leaf cache already serves both accessors.
- **Random**: always regresses for <0, 1, 2> (all cache levels miss, so the extra checks are pure overhead).

ReadAccessor<0> (leaf-only) is unaffected by the fix everywhere — no benefit, no regression. So the fix is a net win where level-1 reuse dominates (leaf-boundary-crossing access on CPU and newer GPUs) and a net cost for scatter-heavy or older-GPU workloads.

For production use with the NEW accessor, the choice of accessor type depends on the workload:

Use case	Recommended accessor
LeafJump / leaf-boundary-crossing access	ReadAccessor<0, 1, 2> — the clearest NEW win (1.6–1.8× CPU-1T)
Stencil / convolution on CPU	ReadAccessor<0, 1, 2> (modest gain, ≈1.0–1.3×)
Stencil on GPU	architecture-dependent: <0, 1, 2> on Blackwell; <0> on Ada/Turing (avoids the 12–20 % regression)
Sequential access	either — roughly a tie
NodeJump / random scatter (CPU)	ReadAccessor<0> — avoids the <0, 1, 2> regression

Per-machine takeaways

- LeafJump is the primary CPU win on all machines: 1.65×/1.48× (Desktop), 1.83×/1.40× (Workstation), 1.58×/1.15× (Dell laptop), 1.63× (Razer Laptop) on CPU-1T/GPU, and 2.55× (MacBook Air M3, CPU-1T; no GPU) for NEW <0, 1, 2> . This is the pattern that directly exercises the level-1 cache rescue: leaf

cache misses but the lower internal node is still cached. (The M3's larger ratio reflects its smaller cache — see the cross-machine scope note.)

- **Stencil GPU results are architecture-dependent.** On SM 12.0 (Blackwell): NEW <0,1,2> improves 10 %. On SM 8.9 (Ada): it regresses 12–14 %, confirmed on **two independent SM 8.9 machines** (Workstation RTX 6000 Ada and Dell laptop RTX 5000 Ada — 0.88× on both). On SM 7.5 (Turing / Razer Laptop): regresses 20 % (0.80×) — the oldest architecture absorbs the extra checks least efficiently. ReadAccessor<0> is neutral in all cases.
- **GPU NodeJump wins on Ada and Blackwell; regresses on Turing: 1.32× (Desktop), 1.48× (Workstation), 1.36× (Dell laptop) vs. 0.76× (Razer Laptop).** With one representative voxel per actual lower-internal node the level-2 cache hit path pays off on newer GPUs; SM 7.5 Turing cannot absorb the overhead.
- **NodeJump CPU-1T regresses for <0,1,2> on the Desktop (0.74×):** level-1 always misses with the real topology pattern; the extra cache checks add latency with no benefit. The GPU still wins (1.32×) because the level-2 hit at the upper-node boundary outweighs check overhead.
- **Random always regresses for <0,1,2>** (0.82–0.95× CPU; 0.79–0.94× GPU across all platforms): all cache levels miss, extra checks are pure overhead. <0> is neutral (0.99–1.02×).
- **ReadAccessor<0> is unaffected by OLD → NEW on CPU** on all platforms and patterns. Its code path (leaf check → root) has no level-1/2 checks to change. On SM 7.5 GPU, <0> also shows no meaningful change (0.81–1.02×).
- **CPU multi-thread scaling** (NEW <0,1,2>, MT vs 1T): scatter/random patterns scale best because multi-threading hides their latency — Random reaches ~16× on the 32-thread Desktop and ~34× on the 64-thread Workstation. LeafJump scales ~9× (Desktop) to ~14× (Workstation). Coherent Sequential access is memory-bandwidth-limited and scales least (~6–8×). The 12-thread Razer Laptop scales ~3–6× across patterns, in line with its 6 physical cores.

Building & running

```
# Configure with the benchmark (and CUDA, for the GPU variant) enabled
cmake -S . -B build -DNANOVDB_BUILD_BENCHMARK=ON -DNANOVDB_USE_CUDA=ON

# Build all eight executables
cmake --build build --target \
    bench_accessor_old bench_accessor_new bench_accessor_singlekey bench_accessor_keyless
    bench_accessor_cuda_old bench_accessor_cuda_new bench_accessor_cuda_singlekey
    bench_accessor_cuda_keyless

# Run — each binary prints results for both ReadAccessor<0,1,2> and ReadAccessor<0>
cd build/nanovdb/nanovdb/benchmark

./bench_accessor_old           # CPU, OLD 3-key accessor (72 B)
./bench_accessor_new           # CPU, NEW 3-key accessor (72 B)
./bench_accessor_singlekey     # CPU, NEW single-key accessor (48 B)
./bench_accessor_keyless       # CPU, NEW keyless accessor (32 B)
./bench_accessor_cuda_old      # GPU, OLD 3-key accessor
./bench_accessor_cuda_new      # GPU, NEW 3-key accessor
./bench_accessor_cuda_singlekey # GPU, NEW single-key accessor (48 B)
./bench_accessor_cuda_keyless  # GPU, NEW keyless accessor (32 B)
```

The accessor behaviour is selected at compile time per target:

Binary suffix	Macros	sizeof ReadAccessor<0,1,2>
_old	-DNANOVDDB_USE_OLD_ACCESSOR	72 B
_new	-DNANOVDDB_NO_OLD_ACCESSOR	72 B
_singlekey	-DNANOVDDB_NO_OLD_ACCESSOR - DNANOVDDB_USE_SINGLE_ACCESSOR_KEY	48 B
_keyless	-DNANOVDDB_NO_OLD_ACCESSOR - DNANOVDDB_USE_KEYLESS_ACCESSOR	32 B

Both accessor types (ReadAccessor<0,1,2> and ReadAccessor<0>) are printed within each binary.

Files

File	Purpose
BenchPatterns.h	Shared access-pattern generation (identical coords for CPU & GPU)
BenchAccessor.cc	CPU benchmark (single- and multi-threaded)
BenchAccessorCuda.cu	GPU (CUDA) benchmark
CMakeLists.txt	Builds the OLD/NEW × CPU/GPU targets